ClausePay: project overview

Summary

ClausePay lets collaborators write their revenue-sharing deal in plain English, then runs it automatically on every PayPal sale. An AI reads the agreement, asks about anything unclear, and writes a small calculation program that works out who gets what. Money only moves after everyone has approved the probationary results and the program has passed automated checks.

It is for small groups who split income: a band, an indie game duo, a podcast, an agency with freelancers. It is a prototype that runs against PayPal's sandbox (test money), and it does not provide any legal or financial advice.

The problem

A revenue split is easy to agree on in conversation and hard to apply correctly month after month. Real deals can contain terms like these:

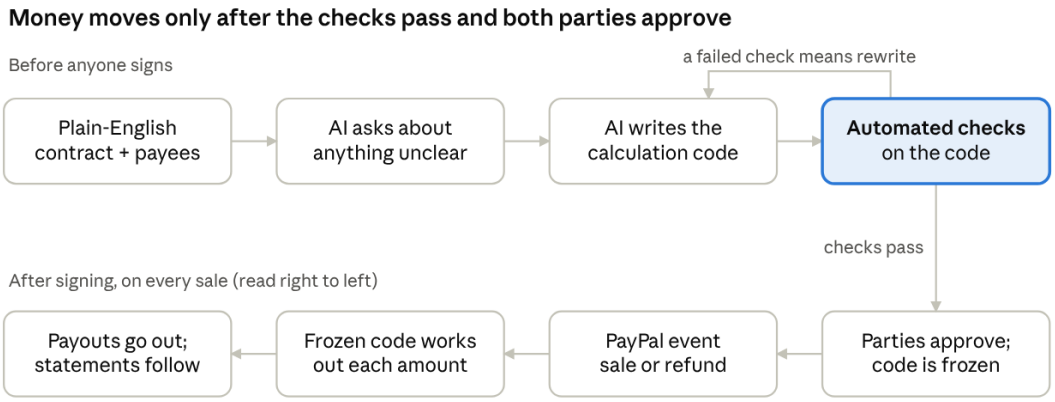
- One person's upfront cost is repaid before profits are shared (recoupment).
- The percentages change once a threshold is reached.
- A slice of every sale is held back for a while in case of refunds.
- Someone has to decide who bears PayPal's fees.

Each term can possibly hide questions the agreement never answered. Is the holdback worked out before or after PayPal's fee? What happens when a refund is larger than the money held back? Ambiguity like this is where collaborators fall out. Today most groups cope with a spreadsheet, a manual transfer each month, and trust.

The gap we are aiming at. Repaying a cost before splitting profits already exists where someone sets the rules by hand in the product. We found nothing that turns a plain-English agreement into rules that run on every sale, checks those rules before money moves, and explains each payout to the person receiving it. We also found nothing aimed at non-music groups, such as game duos, podcasts or agencies, or built on PayPal. On the other hand, the AI tools we found read royalty contracts for label teams, but none of them pays anyone.

How it works

The process has two halves: a setup that ends with everyone's approval, and a routine that runs on every sale.



The automated checks are the gate: a failed check sends the AI back to rewrite the program, and nothing is paid out until both parties approve and the code is frozen.

Details worth knowing

- The payees, meaning names and PayPal email addresses, come from a form and not from the contract text.
- For each sale, the frozen program returns the amounts and a trace of how it reached them. Statements are written from the trace.
- Refunds and disputes arrive as events too. The contract decides where the money is taken from; in our worked example it comes from held-back money first.

A worked example

Alice (art) and Bob (code) sell their game for $21 on PayPal. Across three weeks and a refund, every dollar reconciles. They write their deal like this:

"Take PayPal's fee off first. Hold back 10% for 60 days; refunds and lost disputes are paid from that held-back money. Bob paid $1,000 for the music licence, so he gets everything else until that is repaid. After that, Alice gets 60% and Bob 40%."

What the AI asks before anyone signs

- Is the 10% worked out before or after PayPal's fee?
  Possible answer: after.
- When held-back money is released, does it repay Bob first or go straight to the split?
  Possible answer: repay first, then split.
- If a refund is bigger than the held-back money, who covers the gap?

Possible answer: it comes out of the next payouts, split 60/40.

What happens over time

When	What happened	PayPal fees	Held back	Alice gets	Bob gets
Week 1	40 sales, $840	$40	$80	$0	$720 (all repays his licence)
Week 2	30 sales, $630	$30	$60	$156	$384 ($280 finishes repaying, $104 is his share)
Week 3	One $21 refund, paid from the oldest held-back money	$0	-$21	$0	$0
Day 60	Week 1 holdback released ($80 less the $21 refund = $59)	$0	-$59	$35.40	$23.60
Total	$1,470 in sales	$70	$60 still held	$191.40	$1,127.60

The totals reconcile: $191.40 + $1,127.60 + $60 held + $21 refunded + $70 fees = $1,470 in sales.

What Alice reads after week 2. "You received $156.00. Of $630 in sales, $30 went to PayPal fees and $60 was held back. The next $280 repaid Bob's music licence, which is now fully repaid. The remaining $260 was split 60/40."

Simplifications. This example uses a flat $1 fee per sale to keep the numbers round. It also ignores payout fees and the partial return of fees on refunds from PayPal, all of which we will measure in the sandbox. The real system uses the figures PayPal reports for each sale.

Why the AI writes the engine, and how we keep it safe

The AI writes the calculation program once, the parties approve it, and from then on the same frozen program runs on every sale. We chose this over a fixed menu of rule types because a menu limits which contracts we can handle. The trade-off is that the safety work moves to checking the code before it is trusted.

Who does what

- **PayPal** reports sales, fees, refunds and disputes, and carries out payouts. It does not know why an amount is right.
- **The generated program** takes a sale and the current balances and returns who gets what, plus a step-by-step trace of how it got there.
- **Our own fixed code** talks to PayPal and sends exactly the amounts the program returned. The AI never decides an amount at payout time.
- **The AI, again** reads the trace afterwards and writes each person's plain-English statement.

Checks before the program is trusted

- **Universal rules.** Every contract must satisfy the same tests: payouts plus holdback plus fees equal the sale, no negative payouts, one currency, whole cents, and amounts only for payees on the approved list.
- **Worked examples.** The AI writes numeric examples from the contract, we run the program on them, and the parties confirm the numbers match what they meant.
- **Two independent versions.** A second, separate AI pass writes its own version. We compare both on many random sales, and any disagreement blocks approval.
- **A locked-down runner.** The program is a pure function run in a process with no network, no files and a time limit.
- **Contracts are untrusted text.** The payees come from a form, not from the contract, so a line like "send everything to X" cannot add a recipient.
- **Statements cannot invent numbers.** We check that every figure in a statement appears in the trace.

Cost and fallback. The checking harness is real work, probably about a week, and it is the core of the project. If time runs short, we restrict to a smaller set of contract shapes and still ship.

PayPal products we plan to use

- **Orders API (checkout):** a buyer pays, and we capture the payment. PayPal reports the exact fee on each sale.
- **Webhooks:** PayPal notifies our server of sales, refunds, disputes and payout results. These need a public web address, which we will set up when we host the server.
- **Refunds API:** used to demonstrate clawbacks.
- **Payouts API:** sends each person their amount from the shared business account. It takes a list of recipients and amounts.

Proposed tech stack

Nothing in this stack is decided yet; it is a starting proposal, and the first decision is the backend language. The hackathon accepts any AI tool, as long as PayPal is central to the project.

Layer	Proposal	Reason and status
Backend	Python	PayPal's Agent Toolkit and its examples are TypeScript. Python is the alternative.
PayPal integration	REST API calls: Orders, refunds, Payouts, webhooks	We have already called all of these from a test script.
AI	Claude, through the Anthropic API or Open Weight model like Qwen2.5:32B	Contract questions, code generation, the independent second version, and statements. Undecided.
Running generated code	An isolated process with no network or file access and a time limit	The exact mechanism is still to be chosen.
Data	A ledger of sales, holdbacks, payouts and the frozen contract program	Postgres for the prototype is likely enough
Interface	A web app showing contracts, the ledger and statements, with AG Grid possibly for the ledger	AG Grid is a hackathon sponsor with its own prizes. We have not yet read its entry requirements.
Hosting	Render	Webhooks need a public address, and Render is a sponsor with prize credits. We have not yet read its entry requirements.
Testing	Worked examples as unit tests, plus random sales checked against both program versions	Core to the safety argument above.

Layer	Proposal	Reason and status
Repository	Public on GitHub with an open-source licence	Required by the hackathon rules.

What we have verified so far

A brand-new self-serve PayPal sandbox account can take a payment, refund part of it and pay out two people, with no approval step. We checked this on 26 October 2026 with a test script that runs the whole money loop, and the figures below come from that run. They are sandbox figures, so live PayPal fees may differ.

Test	Result	What it means for the design
Login and permissions	Payouts, refunds, disputes (read and create) and webhooks permissions all granted	Nothing needs approving to start building.
Sale of $21.00	PayPal fee $1.01, net $19.99	The engine must read the fee PayPal reports and never estimate it.
Refund of $5.00	$0.17 of the fee returned, $4.83 deducted from the seller	The fee return is not proportional (a straight proportion would be about $0.24), so the engine uses PayPal's figures. The contract must also say who bears the unreturned fee.
Payout of $3.00 and $2.00	Both succeeded, fees $0.06 and $0.04	Three payouts so far each cost exactly 2%, charged to the sender. The contract must say who ultimately bears them.
Payout of $1.00	Succeeded in about 25 seconds, fee $0.02	Payouts are fast enough for a live demo.

The currency warning. The script reported that PayPal converted currencies on a payout. The business account used so far holds Australian dollars, which is the likely cause. Mixed currencies would make the demo's numbers impossible to reconcile, so we will switch to a

US business account and three US personal accounts, then run the script again to confirm the warning disappears.

Not yet tested

- Webhook delivery. PayPal offers 205 event types, but no webhook is registered, because that needs a public address.
- Creating a test dispute. The permission is present, but we have not used it, and the core project does not need it.
- Payouts to someone without a PayPal account, and returned or unclaimed payouts.
- The contract-to-code pipeline. Nothing is built yet.

Glossary

These are the terms a newcomer is most likely to trip over.

Term	Meaning
Sandbox	PayPal's test environment. It behaves like the real service but uses fake accounts and fake money.
Business and personal accounts	Sandbox accounts of two kinds. The business account receives sales and sends payouts; personal accounts act as buyers and as the people being paid.
Order and capture	An order is a request to be paid. A capture is the moment the money actually moves once the buyer approves.
Gross, fee, net	The sale amount, what PayPal keeps, and what is left.
Payout	Money sent from the business account to one or more people, in a batch.
Payee	A person who receives a share under the contract.
Holdback	A share of each sale kept back for a set time, in case of refunds.
Recoupment	Repaying one person's upfront cost from sales before profits are shared.
Clawback	Taking money back from future payouts, for

Term	Meaning
	example to cover a refund.
Webhook	A message PayPal sends to our server when something happens, such as a sale, a refund or a finished payout.
Trace	The step-by-step record of how the calculation program reached each amount. Statements are written from it.
Frozen program	The approved calculation program, locked so the same code runs on every sale.
Reconcile	Check that every dollar is accounted for: payouts, holdbacks, fees and refunds add up to the sales.